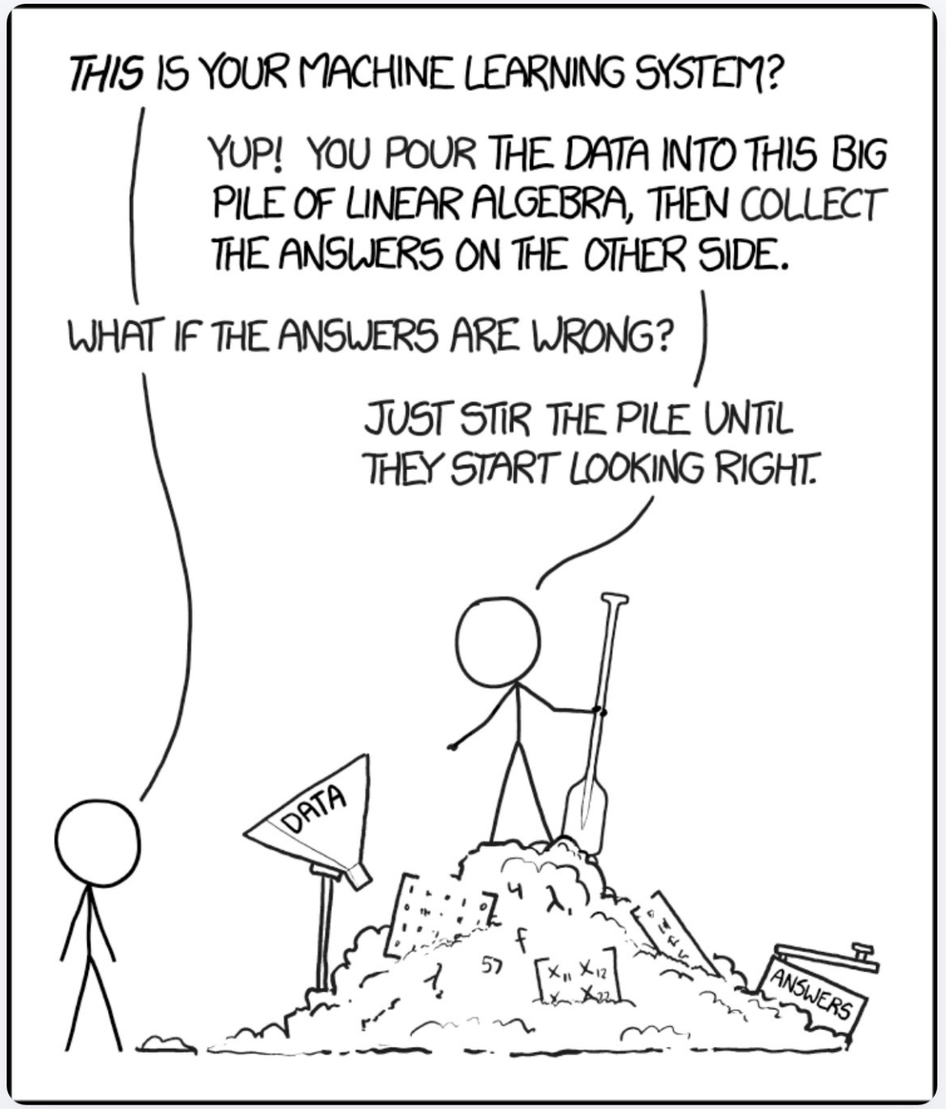


"Machine Learning", xkcd 1838, by Randall Munroe (xkcd.com/1838). CC BY-NC 2.5.

MACHINE LEARNING

Neural networks

Part 2: training a network.

LAST TIME

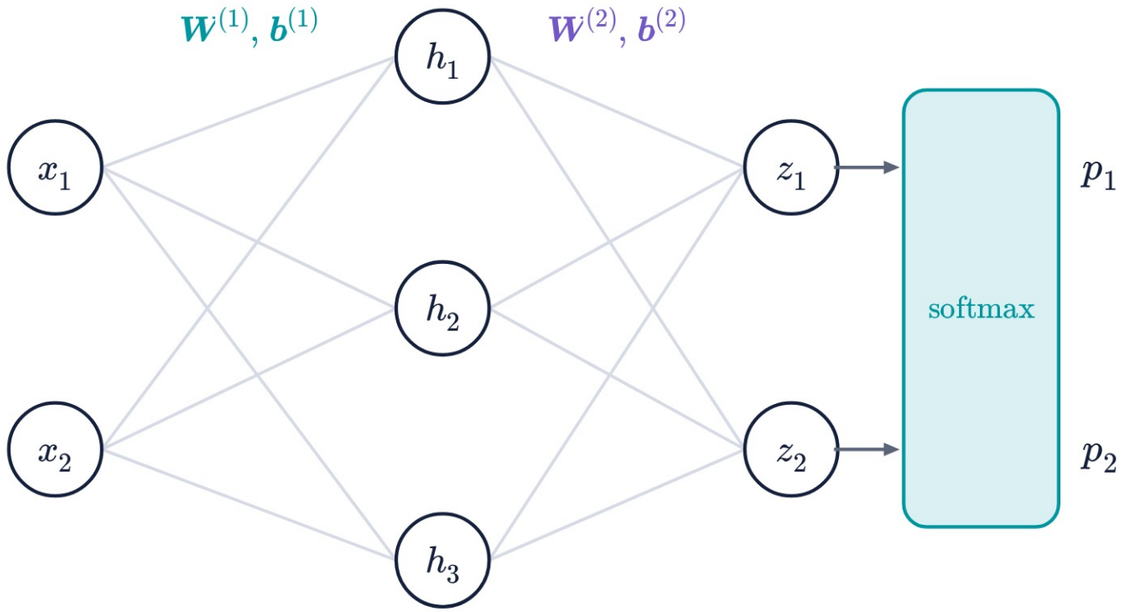
Forward propagation

$$\boldsymbol{h} = \sigma(\boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)})$$
$$\boldsymbol{z} = \boldsymbol{W}^{(2)}\boldsymbol{h} + \boldsymbol{b}^{(2)}$$
$$\boldsymbol{p} = \text{softmax}(\boldsymbol{z})$$

- Train by gradient descent:

  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$

- We need $\partial \mathcal{L} / \partial w$ for every weight w. Where does it come from?



BACKPROPAGATION

The network is one formula

A small network: two inputs, two hidden units, one output. Written out in full:

$$p = \sigma\Big(v_1\,\sigma(w_{11}x_1 + w_{12}x_2 + b_1) + v_2\,\sigma(w_{21}x_1 + w_{22}x_2 + b_2) + c\Big)$$

The loss of one training point:

$$\mathcal{L} = -y\log p - (1-y)\log(1-p)$$

Nine weights, all inside one nested formula. We need $\partial\mathcal{L}/\partial w$ for each one.

The chain rule

- A weight changes the loss through a chain of functions. Multiply the derivatives along the chain:

$$\frac{\partial \mathcal{L}}{\partial v} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial z} \cdot \frac{\partial z}{\partial v}$$

- One output neuron, label $y = 1$ or 0:

$$z = v_1 h_1 + v_2 h_2 + c, \quad p = \sigma(z)$$

$$\mathcal{L} = -y \log p - (1 - y) \log(1 - p)$$

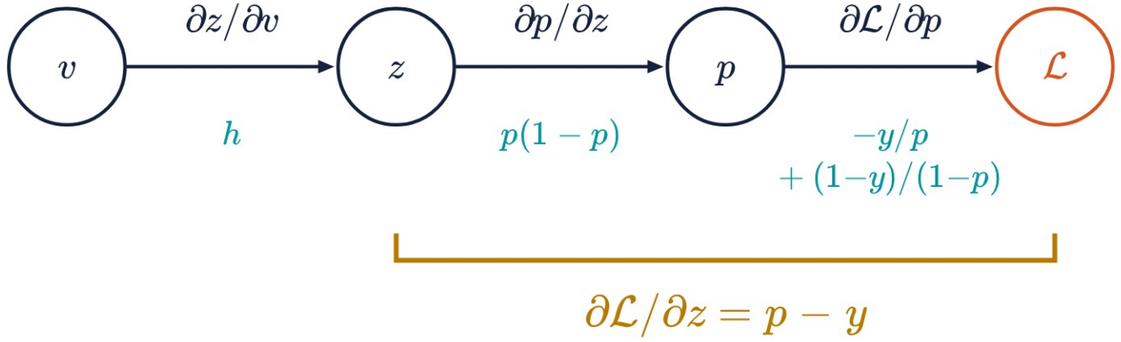
- The two factors:

$$\frac{\partial \mathcal{L}}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}$$

$$\frac{\partial p}{\partial z} = \sigma'(z) = p(1 - p)$$

- Multiply them; it simplifies:

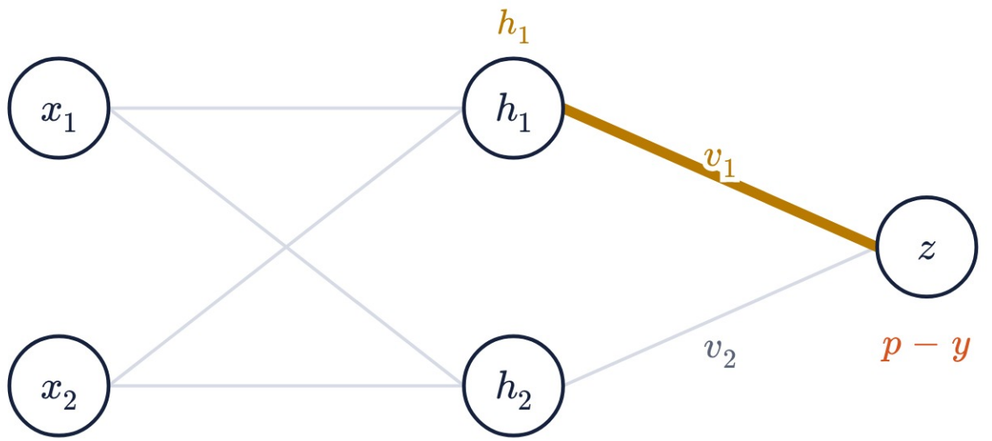
$$\frac{\partial \mathcal{L}}{\partial z} = p - y$$



BACKPROPAGATION

Output-layer weights

- A small network: two inputs, two hidden units, one output.
- The weight v_j reaches the loss through z, and $\partial z / \partial v_j = h_j$:
$$\frac{\partial \mathcal{L}}{\partial v_j} = (p - y)\, h_j$$
- The error at the output times the input to the weight.



BACKPROPAGATION

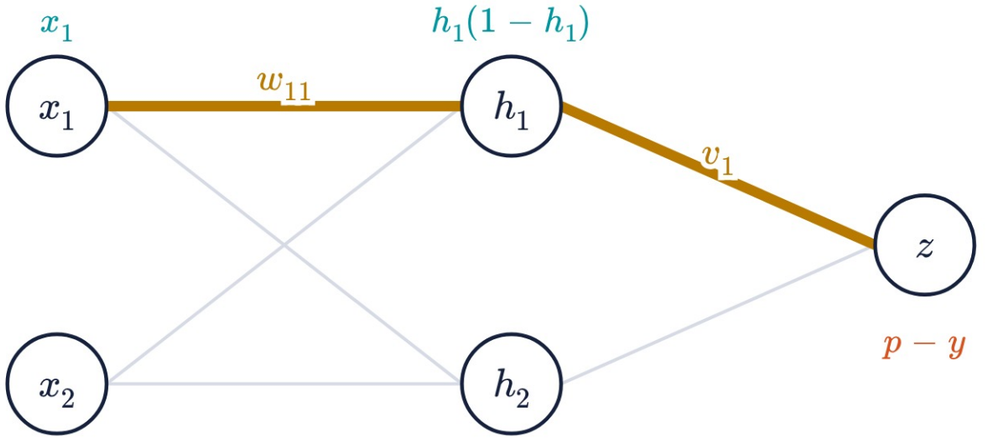
One layer deeper

- The weight w_{ji} connects input x_i to hidden unit $h_j = \sigma(a_j)$, with $a_j = w_{j1}x_1 + w_{j2}x_2 + b_j$.
- Follow the chain from the loss back to w_{ji}:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = (p - y) \cdot v_j \cdot h_j(1 - h_j) \cdot x_i$$

  $h_j(1 - h_j)$ is the sigmoid's slope $\sigma'(a_j)$. With another activation φ, use $\varphi'(a_j)$.

- The first factor, $p - y$, was already computed for the output layer. Reuse it.



Several paths: add them

- With two outputs, h_j feeds both. A hidden weight reaches the loss along two paths.
- Add the paths:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \sum_k (p_k - y_k)\, v_{kj} \cdot h_j(1 - h_j) \cdot x_i$$

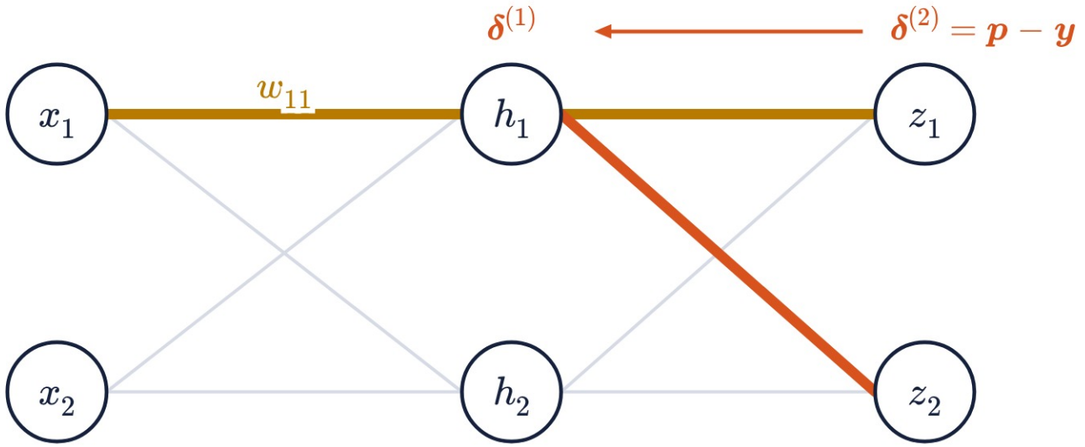
- In matrix form, with $\odot$ the entrywise product:

$$\boldsymbol{\delta}^{(2)} = \boldsymbol{p} - \boldsymbol{y}, \quad \boldsymbol{\delta}^{(1)} = (\boldsymbol{W}^{(2)\top} \boldsymbol{\delta}^{(2)}) \odot \boldsymbol{h} \odot (1 - \boldsymbol{h})$$

- Then:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(2)}} = \boldsymbol{\delta}^{(2)} \boldsymbol{h}^\top, \quad \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \boldsymbol{x}^\top$$

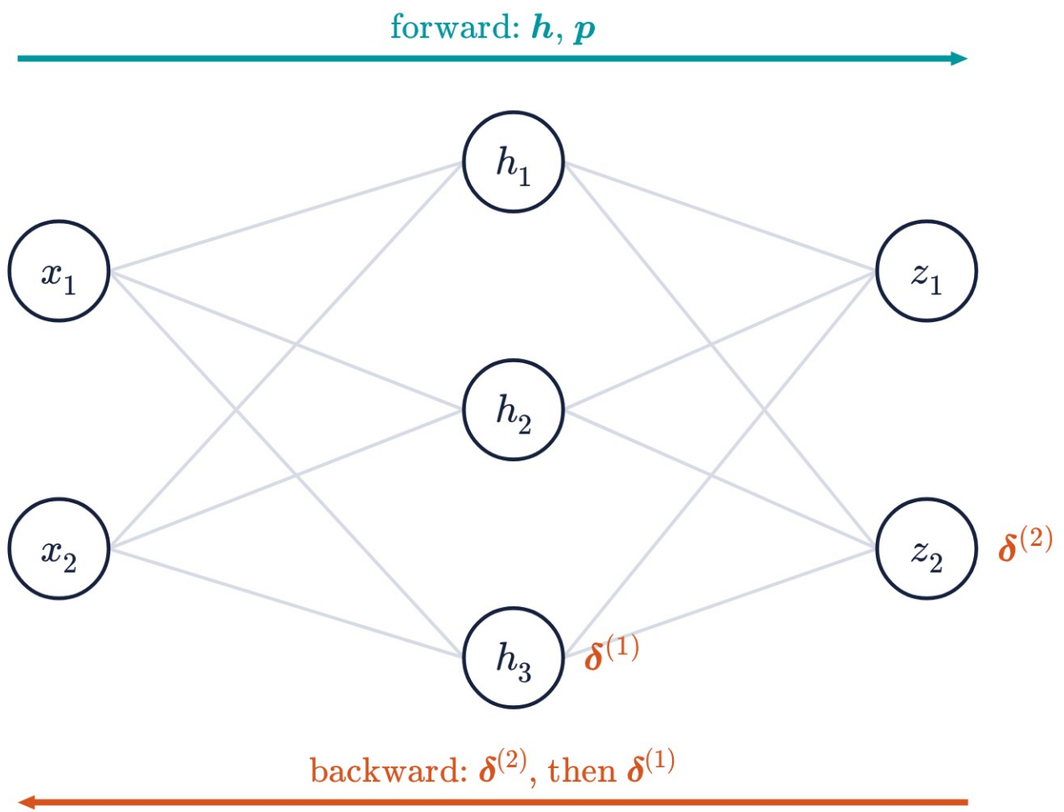
The biases get $\boldsymbol{\delta}^{(2)}$ and $\boldsymbol{\delta}^{(1)}$.



BACKPROPAGATION

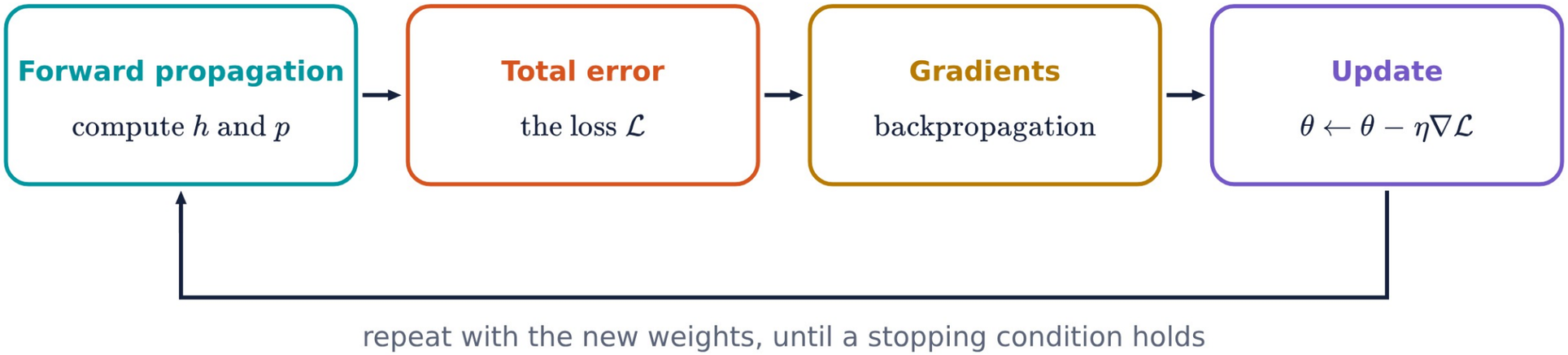
The recipe

1. **Forward pass**: compute and store $\boldsymbol{h}$ and $\boldsymbol{p}$.
2. **Error at the output**: $\boldsymbol{\delta}^{(2)} = \boldsymbol{p} - \boldsymbol{y}$.
3. **Backward pass**:
   $\boldsymbol{\delta}^{(1)} = (\boldsymbol{W}^{(2)\top}\boldsymbol{\delta}^{(2)}) \odot \boldsymbol{h} \odot (1 - \boldsymbol{h})$.
   For sigmoid units. With another activation, replace $\boldsymbol{h} \odot (1 - \boldsymbol{h})$ by $\varphi'(\boldsymbol{a})$.
4. **Gradients**: $\boldsymbol{\delta}^{(2)}\boldsymbol{h}^{\top}$, $\boldsymbol{\delta}^{(1)}\boldsymbol{x}^{\top}$, and the $\boldsymbol{\delta}$s for the biases.
5. **Update** every weight, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\nabla\mathcal{L}$, using the old weights throughout.



PUTTING IT TOGETHER

The training loop



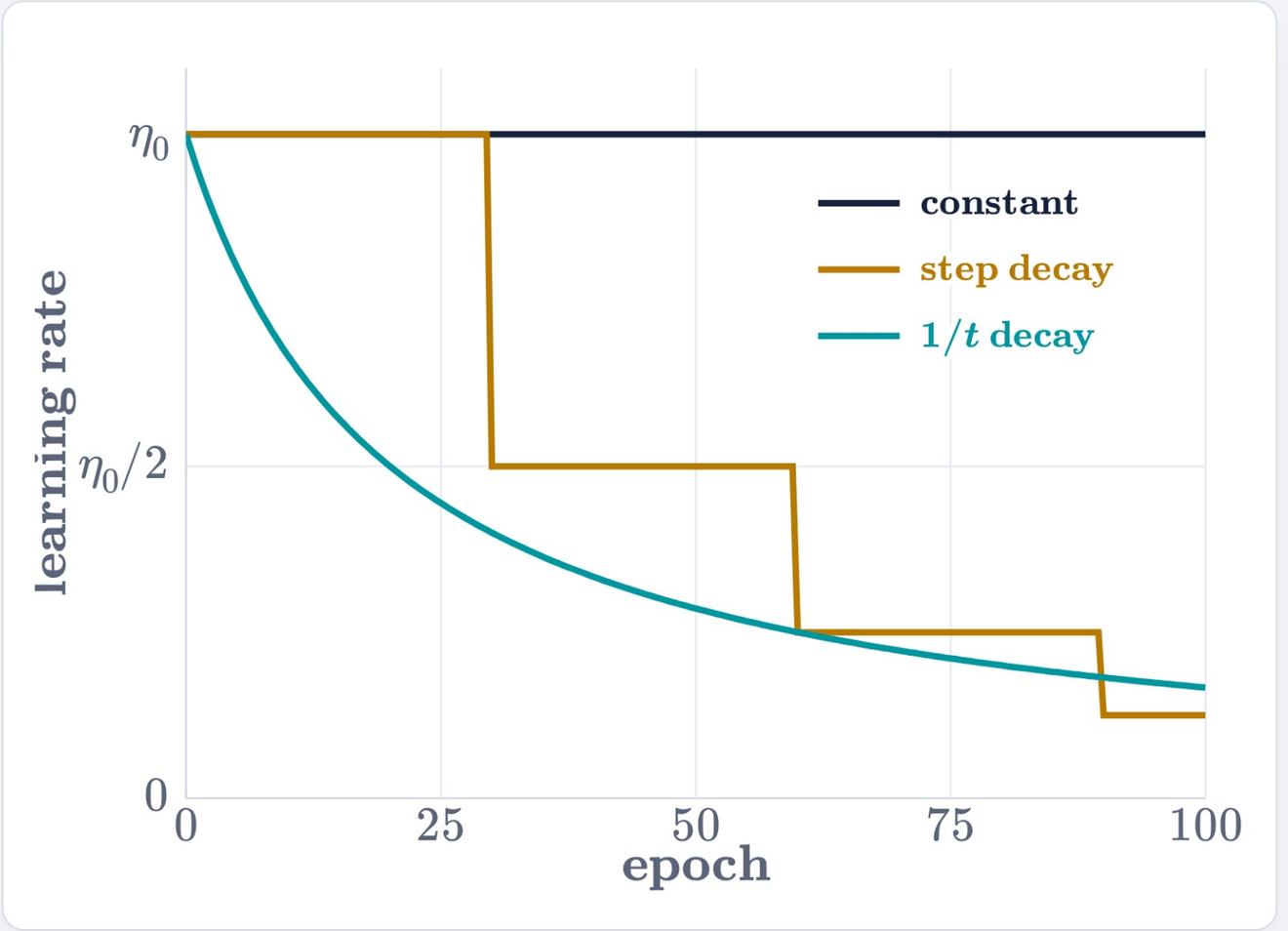
TRAINING IN PRACTICE

The learning rate

- The update:

  $$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}(\boldsymbol{\theta})$$

  Too small: slow. Too large: it overshoots. (Last class.)
- Large steps help early; small steps help near the end.
- A **learning-rate schedule** lowers η as training goes on.



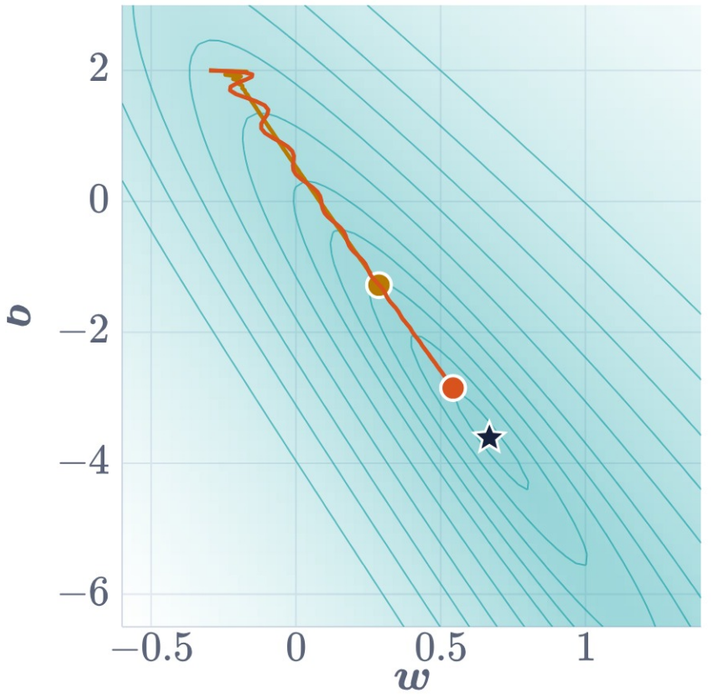
TRAINING IN PRACTICE

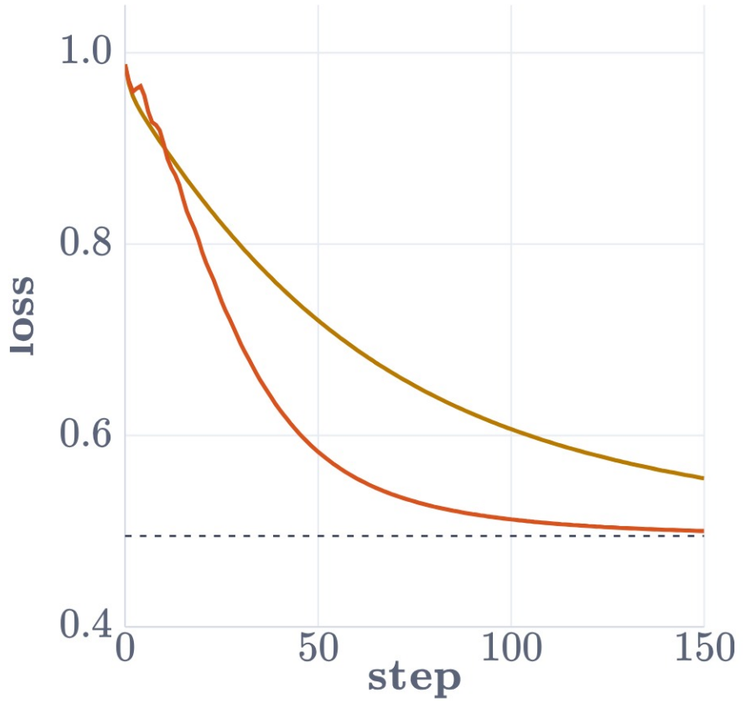
Momentum

- A long, narrow valley: plain gradient descent zigzags across it.
- Keep a running sum of past gradients:

  $$\boldsymbol{v} \leftarrow \beta \boldsymbol{v} + \nabla \mathcal{L}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{v}$$

- The zigzags cancel; the steady downhill part adds up.





STEP	PLAIN LOSS	MOMENTUM LOSS
150	0.555	0.500

▶ step

TRAINING IN PRACTICE

Adam

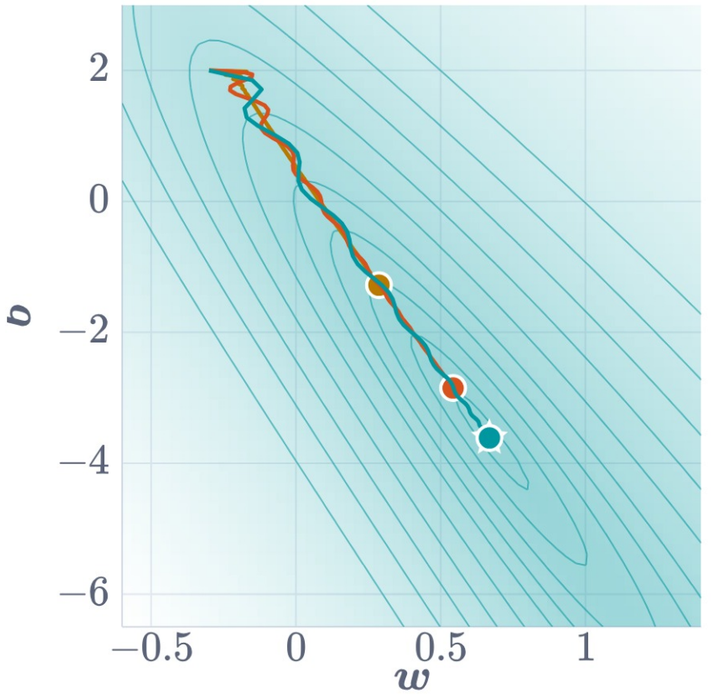
- Momentum, plus a separate step size for each parameter.
- Divide each parameter's step by a running average of its squared gradients:

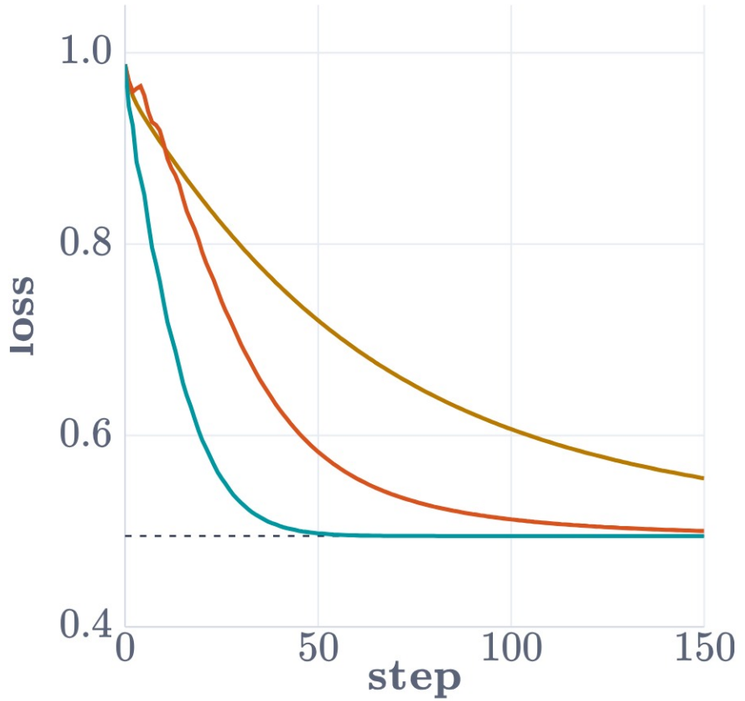
$$\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1 - \beta_1)\, \nabla \mathcal{L}$$

$$\boldsymbol{s} \leftarrow \beta_2 \boldsymbol{s} + (1 - \beta_2)\, (\nabla \mathcal{L})^2$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \frac{\boldsymbol{m}}{\sqrt{\boldsymbol{s}}}$$

- The usual default in practice.





PLAIN LOSS
0.555

MOMENTUM LOSS
0.500

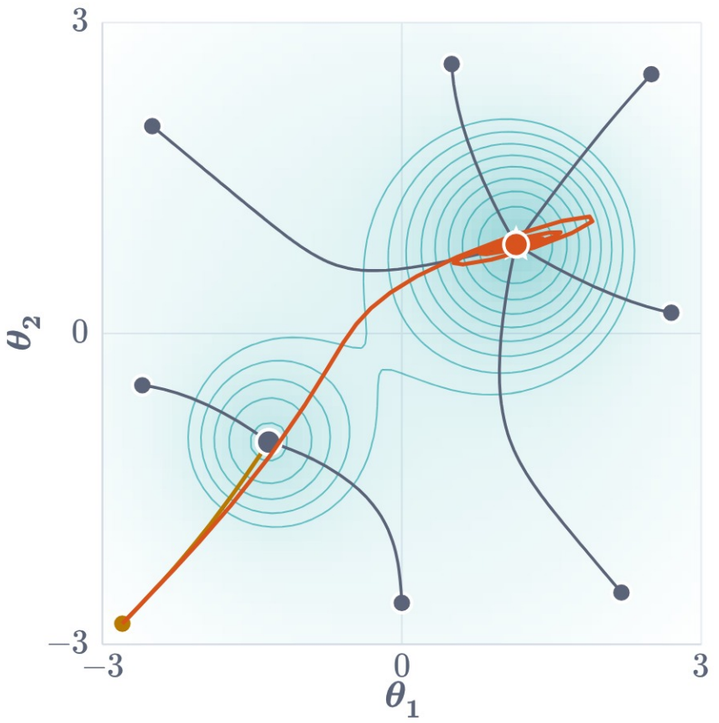
ADAM LOSS
0.495

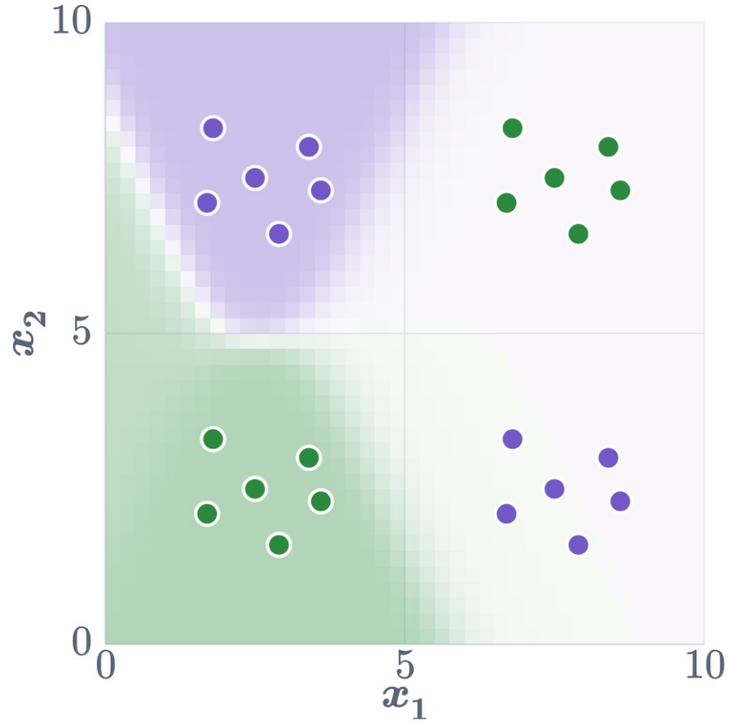
▶ step

TRAINING IN PRACTICE

Local minima

- Gradient descent stops in the valley it starts in.
- Restart from several starting points; keep the lowest loss.
- Momentum can roll through a shallow valley.
- It happens on real networks: the XOR network from a different start stalls.



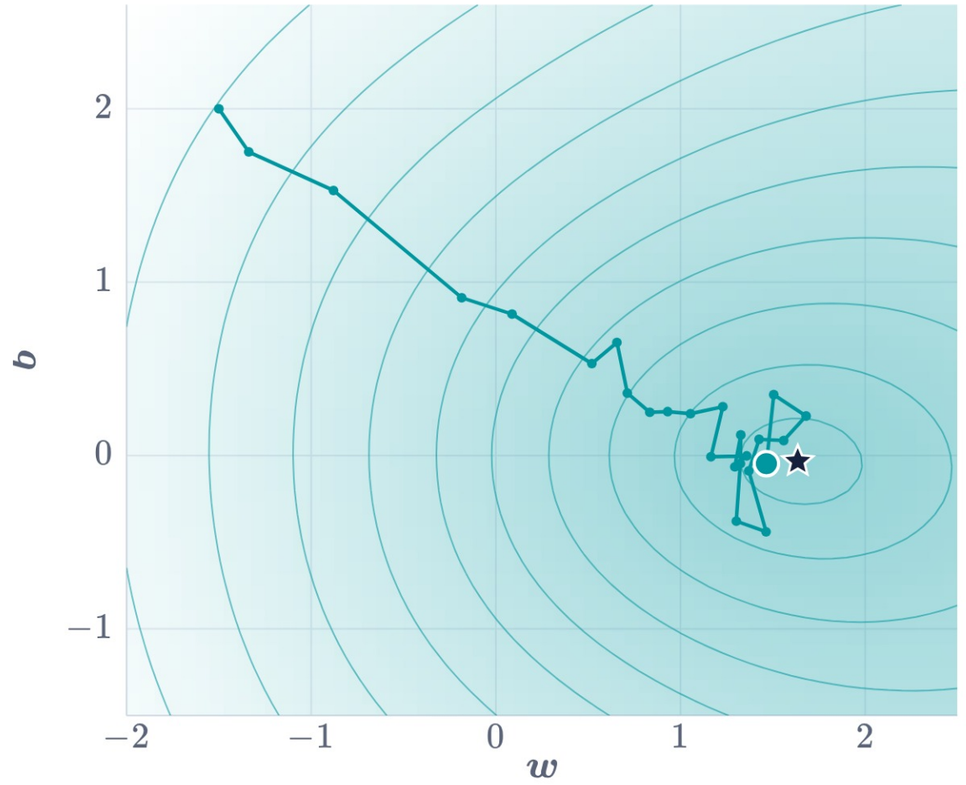


STEPS	LOSS	ACCURACY
4000	0.347	58%

TRAINING IN PRACTICE

How much data per step?

- **Batch**: the gradient from all the training points. One step per pass.
- **Stochastic**: from one point at a time. Many cheap, noisy steps.
- **Mini-batch**: from a few points, here 5. In between.
- An **epoch** is one pass through the training data.



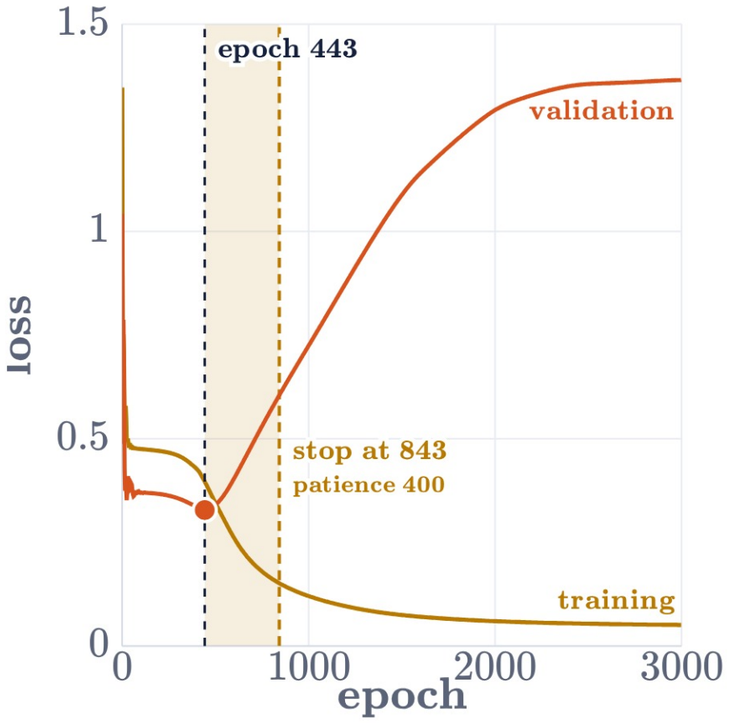
POINTS PER STEP	STEPS PER EPOCH	LOSS AFTER 6 EPOCHS
5	4	0.50

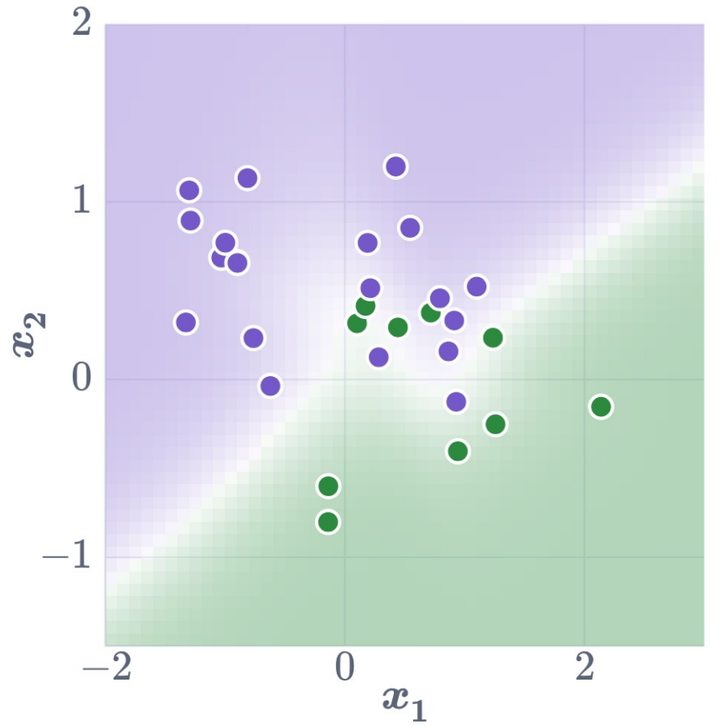
batch | stochastic | mini-batch | ▶

TRAINING IN PRACTICE

When to stop

- A maximum number of epochs; the weights stop changing; the loss is low enough.
- The training loss keeps going down.
- The validation loss goes down, then up: the network starts to overfit.
- **Early stopping**: keep the weights from the epoch with the lowest validation loss.
- **Patience**: stop once the validation loss has not improved for a set number of epochs.





EPOCH	TRAINING ACCURACY	VALIDATION ACCURACY
443	80%	89%

SUMMARY

Neural networks: characteristics

Strengths

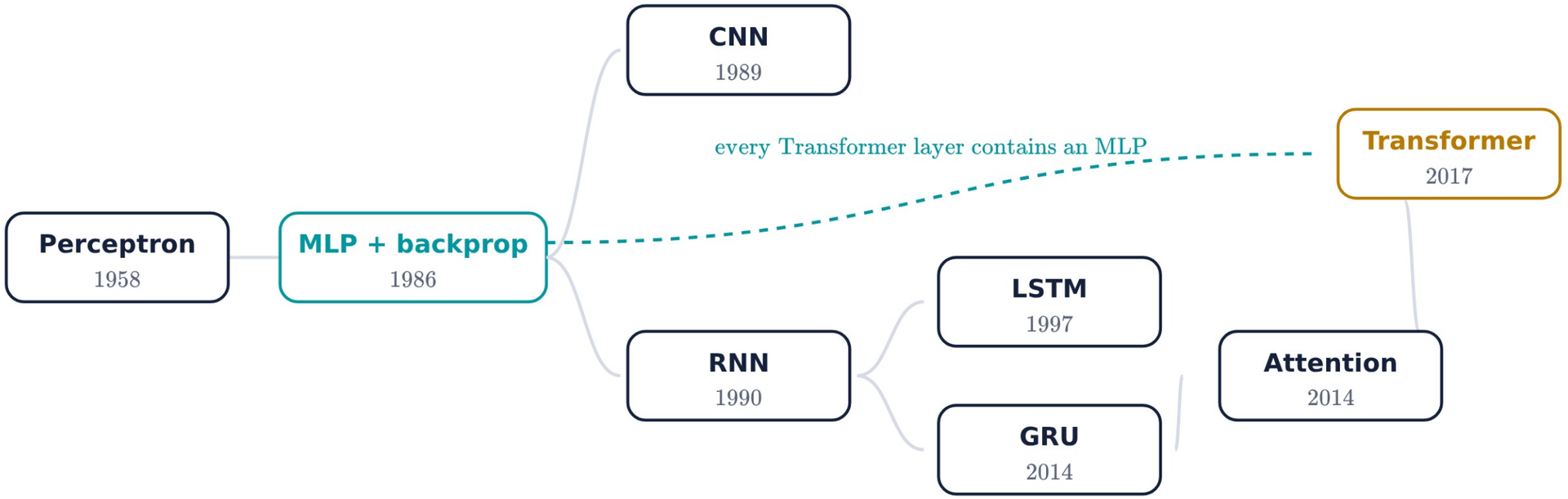
- Very flexible: one hidden layer with enough units can approximate any continuous function on a bounded region.
- They learn their own features, so less hand-made feature design.
- They improve with more data and computing power, and handle many outputs at once.
- Irrelevant inputs can be ignored: training can drive their weights to zero.

Weaknesses

- Non-convex training: the answer depends on the start, with no guarantee of the best weights.
- Many choices to tune: layers, units, η, optimizer, epochs.
- They need a lot of data, overfit small data sets, and are hard to interpret.
- Training is time-consuming: many passes over the data, and many runs to tune the choices above.

WHERE THIS GOES

The family tree



SUMMARY

Takeaways

Backpropagation
The chain rule, from the output back. $\boldsymbol{\delta}^{(2)} = \boldsymbol{p} - \boldsymbol{y}$; each weight's gradient is $\boldsymbol{\delta}$ at its end times the value at its start.

Optimizers
Learning-rate schedules, momentum, Adam.

Batches
Batch, mini-batch, stochastic. An epoch is one pass through the data.

Stopping
Watch the validation loss; stop early.

Thank you.

Questions?